Fuzz Testing of Protocols Based on Protocol Process State Machines

Pengyu Zhao
Information Engineering University
Zhengzhou, China
2635799512@qq.com

Sikang Jiang
Information Engineering University
Zhengzhou, China

Shengli Liu
Information Engineering University
Zhengzhou, China

Long Liu*
Information Engineering University
Zhengzhou, China
* d12_liu@163.com

2024 4th International Conference on Electronic Information Engineering and Computer Science (EIECS) | 979-8-3315-3140-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/EIECS63941.2024.10800590

***Abstract*—Network protocols are important components of network communication systems, supporting interactions and communications among different entities in the network. Their security deserves increased attention and research. Fuzz testing, known for its high efficiency and low false positive rates, has been widely applied in modern vulnerability discovery efforts, yielding significant results in the field of software security. Existing fuzz testing methods primarily focus on local applications, with relatively few studies dedicated to fuzz testing for network protocols. Current fuzzers have several limitations, such as inaccurate state representation, the inability to effectively target critical states for testing, and a lack of effective mutation strategies. To address these issues, this paper presents a protocol fuzz testing method based on protocol process state machines. This approach utilizes state variables identified within the source code to obtain additional process states, establishing a mapping between the state space and the code space to construct a protocol process state machine. Additionally, it involves assigning weights to each state to implement a state selection algorithm that focuses on targeted fuzz testing of critical states. Experimental results from fuzz testing on protocol service programs such as TinyDTLS and ProFTPD show that the tool ZFuzz achieves up to a 30.97% improvement in edge coverage compared to AFLNet and up to a 10.93% improvement compared to NSFuzz. ZFuzz can cover all crashes triggered by the comparative tools and discover 2-4 additional crashes that were not identified by the baseline programs. Moreover, the average time to the first crash trigger is enhanced by 78.10%. These findings suggest that this tool has superior coverage capabilities and vulnerability discovery efficacy.**

Keywords-protocol fuzz testing; vulnerability mining; state variables; state selection

I. INTRODUCTION

In recent years, issues surrounding internet security have emerged frequently, causing significant harm and drawing increased public attention. Against this backdrop, more research has been conducted on network protocols. From their inception, network protocols were designed with numerous rules in mind; however, unforeseen factors can still lead to security vulnerabilities in the extensive process of software development. Additionally, protocol programs possess complex program logic, generating various protocol states and types of vulnerabilities, along with diverse program semantics. The inherent flaws and vulnerabilities in protocol programs can be exploited, resulting in attacks such as remote code execution, denial of service, and information theft, with the number of high-risk vulnerabilities in network protocol implementations increasing year by year [1].

In 2014, researchers discovered the infamous Heartbleed vulnerability in the OpenSSL cryptographic library [2]. In 2017, the EternalBlue vulnerability in Microsoft's Server Message Block (SMB) [3] protocol was exploited by attackers, leading to widespread infections from the WannaCry ransomware [4]. In 2020, 360CERT discovered that attackers could send DNS query requests to malicious name servers, causing denial of service for recursive servers and specific domain name servers [5]. These protocol vulnerabilities can be exploited by attackers, causing servers to behave unexpectedly and resulting in denial of service attacks and remote execution attacks. Therefore, vulnerability discovery research on network protocol implementations is essential.

Vulnerability discovery is a critical method for detecting software security issues, and the technical development of vulnerability discovery for network protocol programs has undergone many stages, evolving from classic black-box fuzz testing to feedback-based gray-box fuzz testing, and now to hybrid testing techniques that incorporate program analysis. Fuzz testing involves introducing random input data into the target program and monitoring for anomalies such as crashes and assertion failures to identify potential security vulnerabilities, including memory leaks. The earliest popular black-box fuzz testing tools for protocols include Peach [6] and Boofuzz [7], which require the specification of the network protocol message format and the design of test case generation strategies. With the development of intelligent feedback fuzz testing, a number of gray-box fuzz testing tools, such as AFL, have gained more attention. For protocol fuzz testing, the primary approach has been to modify AFL [8] to accommodate the interaction characteristics of network protocols and to study general instrumentation techniques to adapt AFL, with a typical tool being AFLNet [9].

However, most existing gray-box fuzz testing tools still have shortcomings in their protocol state expression schemes. AFLNet uses response codes in response messages to signify service states, but this approach does not yield completely reliable states. StateAFL uses memory information to represent the program's service state, but directly mapping complex memory states to service states can lead to issues. NSFuzz proposed a lightweight state representation scheme, but it did not link the state space with the code space. Although some existing tools have considered protocol states as important information, they lack an organic integration of coverage-guided strategies to jointly guide fuzz testing and remain relatively independent without correlated analysis.

To address the current issues in protocol fuzz testing, this paper proposes a protocol fuzz testing method based on protocol process state machines. First, we perform static analysis on the protocol program code, using heuristic rules to identify variables related to protocol state transitions, followed by instrumentation tracing to obtain more levels of states and process information. Next, we construct a protocol process state machine that organically maps the state space to the code space, assigning weights to states to increase the exploration probability of valuable states. This approach was tested on protocol service programs such as TinyDTLS and ProFTPD, using AFLNet and NSFuzz as comparative tools. The experimental results not only showed improvements in edge coverage—up to 30.97% higher than AFLNet and 10.93% higher than NSFuzz—but also yielded more crash triggers, uncovering 2-4 additional crashes not identified by the comparative tools, with an average 78.10% improvement in the time to first crash trigger, demonstrating the effectiveness of the proposed tool.

II. Background

When performing fuzz testing on network protocols, the fuzzer acts as a client, communicating with the program through a specific port and protocol, generating and sending data packets, and receiving and processing responses. After processing the request, the internal state is updated, and the result is returned. Network protocol programs need to interact with the client to infer state transitions, and deeper code can only be triggered by subsequent test cases when specific preconditions are met. The program uses network sockets as an I/O interface, communicating with entities multiple times and having its own state, making gray-box fuzz testing challenging.

Currently, there are two main approaches to fuzz testing network protocols. The first is black-box fuzz testing, which involves constructing network data packets and sending them to the target port of the program under test. However, this approach lacks feedback information to guide the testing process, making the fuzzer relatively blind and often ineffective. Representative tools for black-box fuzz testing include Peach and Boofuzz, which use custom templates to generate test data packets and send them to the target port, identifying potential issues based on response times and other information. However, writing protocol primitives requires significant human intervention and expertise, and the lack of feedback information to improve test case quality results in a large number of ineffective test cases, making the testing process highly random.

SNOOZE [10] is a protocol-state-driven black-box fuzz testing method that takes user-defined test schemes and XML language descriptions as input, generating test cases and detecting issues. KiF [11] is a stateful fuzz testing model for SIP protocols, consisting of protocol syntax fuzz testing and state evaluation components. AutoFuzz [12] relies on proxy services to obtain protocol interaction traffic, building a behavior model based on the protocol's state machine, and then traversing the service state space by modifying messages. AspFuzz [13] describes application-layer protocol specifications as protocol definitions, changing the sending order of generated message sequences to perform state-level fuzz testing. PULSAR [14] combines protocol reverse engineering and simulation techniques to analyze unknown protocol network traffic, inferring protocol formats and state models, and guiding the fuzzer to traverse the finite state machine model.

The second approach is gray-box fuzz testing, which is represented by AFLNet [15]. AFLNet splits traditional file-based test cases into message chains, performs message-level mutations on data packets, and sends them through network communication. AFLNet analyzes the response packets from the network server, extracts its internal state, and implements testing for specific states. This approach also obtains the code coverage of the protocol under test, improving the effectiveness of the testing.

Compared to PULSAR, AFLNet avoids the need for protocol reverse engineering before testing. SGFuzzer [16] also attempts to represent protocol states using response codes, analyzing source code to obtain offline state variables and building a protocol state transition tree at runtime. StateAFl [17] uses memory state to represent service states, performing online variable analysis on the protocol under test and then collecting and building state models. SGFuzzer enhances the effectiveness of test cases by pre-defining protocol specifications. SNPSFuzzer [18] proposes a snapshot-based testing method, storing program context information at specific locations and restoring context information when testing related states, improving testing speed.

A. Problems

Due to the inherent statefulness of protocols, the reactions to the same input can differ based on the current state, and certain vulnerabilities can only be triggered when specific states and messages are satisfied. Efficient gray-box fuzz testing of network protocols not only requires a reasonable description and representation of the protocol's own states but also necessitates ongoing attention to the transitions of program states. However, due to the complexity of network protocol programs, existing protocol fuzz testing tools cannot effectively explore the protocol state space.

AFLNet proposes a stateful gray-box fuzz testing approach for protocols, utilizing response codes in response messages to represent service states. It then infers state models from sequences of response codes to guide subsequent fuzz testing processes. StateAFL, on the other hand, represents program

service states using memory information, conducting online variable analysis on the protocol being tested to collect states and build state models. In each round of network interactions, StateAFL stores program variables into an analysis queue and analyzes this queue after executing test cases, subsequently updating the state model.

Regarding state representation, the response code scheme proposed by AFLNet embeds special codes within the response messages, but not all protocols meet this condition. StateAFL also points out the limitations of this scheme by using program memory states to represent protocol states; however, mapping positions using sensitive hashes can lead to inaccurate representations. SGFuzz has also suggested using state variables to represent protocol states and constructed a state transition tree to represent the state space explored by the service program. However, SGFuzz's representation of state variables is too coarse-grained and lacks response filtering. NSFuzz, through a study of representative network service programs, proposed a lightweight state representation scheme: the network service program has an event handling loop that processes messages, where heuristic rules are used to identify state variables related to the intrinsic semantic information of the protocol, achieving higher accuracy and interpretability.

In addition to the challenges of state representation, existing tools also face issues with feedback information being too independent, resulting in the inability to focus on key states. Work based on AFL's mutation concept utilizes coverage feedback to guide mutations, enabling the automatic construction of state machines without requiring prior knowledge of the protocol. However, the unknowns regarding protocol states during this process can hinder the acquisition of complete feedback, causing state feedback and coverage feedback to be relatively independent when used, which prevents associative analysis and organic integration. Moreover, the scheduling of seeds is staged, with state selection occurring before seed selection. Existing work has failed to combine the state space with the code space and lacks research targeting key states.

To address the aforementioned challenges in protocol fuzz testing, this paper proposes a protocol fuzz testing method based on protocol process state machines. By analyzing the source code of protocol programs to obtain state variables, a more fine-grained state representation scheme can be derived. Then, by using inter-process information to construct state machines, the state space and code space can be integrated, jointly guiding the fuzz testing process effectively.

III. Approach

As mentioned above, existing protocol fuzz testing solutions face issues such as the inability to reasonably represent the states of protocol programs, the lack of continuous focus on key protocol states, and the randomness of mutation strategies. To address these problems, this solution first identifies program variables that can represent the states of protocol programs and reflects the program's state transitions by tracking changes in these variables. It constructs a correlation between the program space and the protocol state space, designing a fine-grained protocol process state machine. Additionally, it designs a state selection scheme and a mutation strategy that focuses on information perception for key states.

A. Framework Design

The protocol fuzz testing framework designed in this paper is illustrated in the figure 1, which mainly includes modules for the identification and tracking of protocol state variables, the construction of protocol process state machines, and state selection. ZFuzz first conducts static analysis on the source code to obtain the event handling loop and state variables, and then instruments the state variables to track their subsequent changes. By tracking the state variables, detailed protocol states and inter-process information are obtained, constructing a protocol process state machine that integrates the state space and code space. We assign weights to the states to facilitate the selection and in-depth exploration of key states, adopting the mutation strategy used in AFLNet.

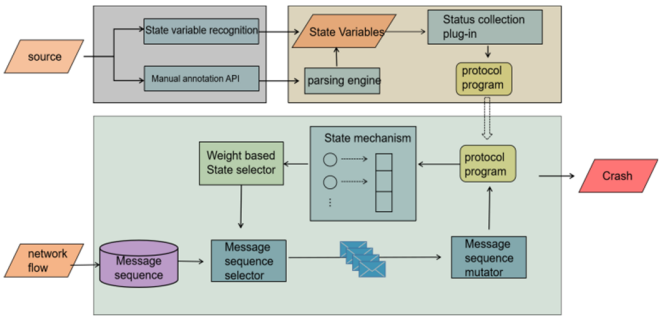

Figure 1. Framework design diagram.

As shown in Fig 1, detailed introduction is as follows:

- Protocol State Variable Identification and Tracking Module. This module processes the source code of the target program through automated analysis and manual annotation of APIs to obtain the event handling loop and state variable list for the protocol program. In the variable tracking phase, the instrumentation engine first parses the manually annotated information, producing corresponding output. At the same time, it instruments the target program based on the state variable list. The instrumented protocol implementation undergoes subsequent fuzz testing, allowing for the acquisition of state feedback information and coverage feedback information during execution.
- Protocol Process State Machine Construction and State Selection Module. Existing gray-box fuzz testing tools for protocols often contain only simple state information in their protocol state machine models. For instance, AFLNet uses response codes from response messages, while SGFuzz utilizes enumeration-type variables as state variables but lacks filtering. This results in existing state machines suffering from coarse states, inaccurate representations, and a focus solely on reachability. Therefore, this state machine construction module is designed to expand the protocol process state machine structure, mapping the program space and state space, and collaborating with protocol state variables to achieve finer state design and enhanced inter-process information. In the abstraction layer of

the protocol program, upper-level regions can directly or indirectly impact the exploration of lower-level regions. During the protocol fuzz testing process, a reasonable state should be considered first, followed by the scheduling of corresponding seeds. Thus, this paper designs a state selection algorithm to calculate the weights for protocol states, using heuristic algorithms to assign energy to key states.

- State and Information Perception-Based Variation Module. In the abstraction layer of the protocol program, upper regions can directly or indirectly influence the exploration of lower regions. During the protocol fuzz testing process, the first consideration should be reasonable states, followed by scheduling the corresponding seeds. Thus, this paper designs a state selection algorithm to compute weights for protocol states, utilizing heuristic algorithms to allocate energy to key states. Subsequently, a message pool of interest is constructed for specific states, and messages from specific locations are extracted according to the state and information perception scheme. A value evaluation calculation formula is used to score these messages, guiding the subsequent seed variation process.

B. Protocol state variable identification and tracking module

The protocol program exhibits clear phase characteristics during its operation. Taking a TCP-based server protocol program as an example, its lifecycle can generally be divided into three stages: service initialization, service processing, and cleanup. Research primarily focuses on the service processing stage, where request messages are handled and responses are generated through network connections. Specifically, this phase typically utilizes an event processing loop. Within this loop, sockets are employed to read request messages, followed by parsing and transaction processing, after which response messages are written back to the socket for communication. The protocol service program continuously executes this loop to interact with clients until the connection is terminated.

For stateful protocol programs, multiple state transitions occur during event processing. Through extensive analysis, it has been determined that developers use enumeration types or integer constants to identify protocol states. Additionally, these variables exist as global variables or struct member variables within the protocol program.[11]

Consequently, these key variables within protocol programs can effectively represent the protocol states. For instance, when Bftpd receives a PASS request message, it first checks whether the current state allows for processing that message. NSFuzz will only pass the password field from the request message as a parameter to the processing function after successfully passing the state check; otherwise, it will directly return an error response. After a user successfully logs in with the correct password, Bftpd updates a variable called `state`, which is a global variable representing the current state, and the updated value of this variable is of the enumeration type. Notably, this variable is consistently read or updated within the event processing loop. Moreover, using program variables to represent protocol states is more reasonable than relying solely on response codes. For example, Bftpd returns a response code of 503 in different states, which does not effectively convey the program's process or distinguish between states.

1) Event processing loop recognition.

In the process of identifying the event handling loop, it is necessary to distinguish it from the loops of other network service programs while also excluding the interference of nested loops within the event handling loop.

First, when the network protocol program completes initialization and enters the service processing phase, breakpoints are set at the input system calls in the program. When subsequent messages trigger these breakpoints, the current function call stack information of the protocol program is saved as auxiliary information for loop identification. During the identification process, all loop structures that include I/O operations are collected, and a mapping relationship is established with their corresponding functions.

Then, based on the stack backtrace information, the functions are matched layer by layer from the bottom of the function call stack. The first matched function's loop structure is taken as the event handling loop. This is because the call stack backtrace information only records function calls during the service processing phase, which helps avoid interference from loops in the initialization phase. Moreover, the bottom-up function matching identifies the outermost loop, excluding the influence of nested loops. Therefore, the call stack backtrace information when the protocol program reads messages can be used to identify the event handling loop.

2) State variable recognition.

Due to the lack of runtime information in static analysis, there is a possibility of false positives during the identification of state variables. To address this issue, a summary of the characteristics of state variables in protocol programs is undertaken, utilizing heuristic rules to reduce instances of misidentification.

- Limiting Scope to the Event Processing Loop: Since operations on state variables in protocol programs occur exclusively within the event processing loop, the analysis is confined to the program variables within that loop. This narrows the focus and reduces the chances of false positives.
- Recording Loaded and Stored Variables: State variables in the context of the event processing loop or message handling functions are specifically read or written. Therefore, the approach only records variables that are loaded (read) or stored (written), ensuring relevance.
- Focusing on Global and Struct Member Variables: Protocol state variables are typically global variables or member variables within structures and are often assigned constant values. Hence, only global enumeration or integer variables that are assigned constants, as well as respective member variables, are retained in the identification process.

Upon completion of state variable identification, each variable is assigned a unique string identifier, which is output in a list format for subsequent tracking module use. However,

for protocol programs developed using event notification libraries, the event processing loop is often encapsulated within library functions. To assist with this, the proposed scheme includes a manual annotation module. By utilizing annotation APIs to annotate key information, this allows for a more granular identification of state variables and enables state-sensitive operations at different levels. Furthermore, it permits precise annotation of state variables based on protocol knowledge, improving overall accuracy and functionality in identifying relevant state variables.

3) Status tracking collection

To enable real-time transmission of state variable values to the fuzz tester, instrumentation is applied to the write operations of state variables in the source code during the compilation phase, facilitating subsequent state collection. Additionally, the values written to state variables are mapped to a shared memory space between the fuzz tester and the protocol program under test, referred to as share_state. The specific mapping calculation method is as follows:

$$shared\_state[hash(var\_id) \oplus cur\_store\_val] = 1 \quad (1)$$

$$shared\_state[hash(var\_id) \oplus pre\_store\_val] = 1 \quad (2)$$

In this process, `var_id` represents the string identifier for the state variable, which will be subjected to hash processing subsequently. The hash value is then XORed with the new value to be written, and the resulting XOR value is used as an index to update the shared memory block. Moreover, this module can calculate the index based on the previous value of the state variable to retrieve the corresponding value from the shared memory.

Different state variables will generate different hash values, corresponding to distinct areas in the shared memory. Every time a state variable's value changes, it triggers a modification in the shared memory, thereby allowing for a more detailed recording of the state transition process within the protocol program. The instrumented program will serve as the final version for subsequent processing, including the construction of state machine models and mutation operations.

C. Protocol Process State Structure Construction and State Selection Module

1) Construction of Protocol Process State Structure.

In the current process of state machine construction by gray-box fuzz testers, simple feedback codes or unprocessed program variables are typically used, lacking a connection to the program's code space. These two types of feedback information are relatively independent and do not effectively collaborate to guide the fuzz testing process. This module expands the structure of the state machine M and designs an enhanced mapping relationship between the protocol state machine's state space and the program space, utilizing program variables to represent fine-grained program states and increase the amount of inter-state process information.

First, the state machine M is extended into a directed graph structure <S, E, ∑:S→{re_mes}>, where ∑ maps messages to corresponding states. If the program state is Vi before sending re_mes, the corresponding changing program variable var_id will be added to V, and the edge v→var_id will be added to E, along with the mapping relationship into ∑.

Next, the state nodes are expanded to <depth, pnode, statetrans_paths, covered_bits, selscted_time, fuzzed_times>. Here, depth∈N represents the depth of the state; pnode∈V indicates the predecessor state node; statetrans_paths∈N represents the number of state transition paths explored at this state; covered_bits∈M denotes the bitmap of program coverage at this state, where M is a two-dimensional matrix; selected_times∈N indicates the number of times the state has been selected; and fuzzed_times∈N represents the number of times the state has been tested.

In the protocol state graph, the current state node vi∈V, has a depth recorded as the distance d(v, v0) to the entry node v0, based on the observation that state nodes with greater depth values are harder to explore. According to the state variable collection, the shared memory block shared_state is obtained. Each time the fuzz tester receives instrumentation feedback from the protocol under test, it records the current program variables in shared memory, organizes them for hashing, and uses this hash value to represent the overall state of the current protocol program. Each hash value is placed in a queue, and as requests are continually sent, a set of state sequences state_sequence is obtained after the test cases are executed. By comparing different state sequences, the changing shared memory blocks shared_state corresponding to the state variable identifiers var_id can be identified. The function unique is then used to compare the number of different hash values, yielding statetrans_paths as unique(hash(var_id)).

The statetrans_paths indicates the number of state transition paths that can be discovered when selecting a certain state vi. A larger value signifies that more transition paths can be explored from the current state, indicating a higher degree of association with other states and suggesting that this state is more important and worthy of closer attention. The covered_bits represent the edge coverage bitmap achievable when exploring this state, with each state retaining its own independent bitmap. During fuzz testing, the program coverage after executing test cases in this state is recorded, and the bitmap coverage for each node in the state machine is calculated. As testing progresses, the mapping between the protocol program space and the state space is continuously refined.

From the above, it can be concluded that for a known protocol state vi, the coverage covered_bits that can be triggered by this state is obtainable. A larger value indicates a larger corresponding program space, enhancing the exploration value of this state. By supplementing the number of times the state has been selected selected_times with the number of times it has been tested fuzzed_times, a balance between state exploration and utilization can be achieved, leading to more rational testing.

2) State Weight Calculation.

This article utilizes state variables to obtain a more detailed protocol state. However, during the fuzz testing process, not all states hold equal value; the testing needs to focus on identifying key program states and testing them in the shortest time possible. Additionally, existing mutation methods are

insensitive to the structure of messages, leading to a large number of ineffective test cases. To obtain better test cases, a state and message-aware mutation strategy has been designed.

To identify key states, a method for calculating state weights has been developed, using heuristic algorithms to allocate resources to these key states. The weight calculation is based on the following understanding:

- The deeper the code in the network communication layer, the harder it is to test. States have forward dependencies, and as the level of communication deepens, the required message sequences also become more difficult to construct, leading to challenges in achieving thorough testing and a higher likelihood of hidden vulnerabilities.
- States located at critical positions in the state machine are associated with more state transition paths. Exploring such states is more likely to trigger unknown coverage.
- The larger the program space affected by a state, the more complex the implemented functionality will be. The vast program space and complex code implementations increase the difficulty of comprehensive testing, so these states should receive greater attention.
- In the process of exploring program states, it is essential to consider the exploration of new states to avoid an incomplete state machine structure.

Based on the above principles, this article proposes a formula for calculating state weights. For state v_i, the weight calculation formula is:

$$w_i = \frac{\text{depth}_i \times \text{statetrans\_paths}_i \times \text{covered\_bits}_i}{\lg \text{selected\_time}_i + \lg \text{fuzzed\_time}_i} \quad (3)$$

The calculated weight serves as the probability of selecting a state. The deeper the position of the state, the larger the associated paths, and the greater the corresponding program space, the higher the weight value. At the same time, the states that are selected and tested more frequently will have corresponding limitations. To facilitate calculations for high-magnitude selection times, a degradation process is applied to their magnitude. More resources are allocated to key states for further exploration. Additionally, this article continues to use the seed selection and mutation strategies in AFLNet for subsequent fuzzing.

IV. Evaluation

In this paper, experiments were conducted on a server with an Intel® Core™ i7-8700 K @ 3.70GHz CPU and 32GB of memory. Three metrics were proposed to test the performance of the tool, namely: throughput, the number of code edges covered, and the number of crash triggers. This paper selects widely used protocols as benchmarks for testing, including RTSP, FTP, and DTLS, as shown in Table I. The service implementation programs for these protocols have previously been selected by relevant fuzz testing work and subjected to corresponding security analysis.

TABLE I. Target Service Information

Target Service	Type	Git commit id
PureFTPD	FTP	C21b45f
ProFTPD	FTP	4017eff8
Live555	RTSP	Ceeb4f4
Exim	SMTP	38903fd
Tinydtls	DTLS	06995d4

A. Code coverage capability assessment

We use AFLNet and NSFuzz as benchmark programs to test the selected target protocol programs, obtaining edge coverage information to evaluate the tool. Specifically, the experiments involve a 24-hour test comparing the benchmark tools with the tool implemented in this paper, ZFuzz, collecting code branch coverage data from the same protocol program under test. The table below presents the final experimental results.

TABLE II. Branch coverage Situation

Target Service	AFLNET	NSFUZZ	ZFuzz
PureFTPD	983	1035	1152
ProFTPD	4650	4783	5182
Live555	2776	2803	2895
Exim	3588	3621	3770
Tinydtls	465	549	618

The data from table II indicates that under the same testing time and experimental conditions, ZFuzz achieves higher code coverage across all targets. Code coverage is a commonly used metric in fuzz testing, indicating how much code was explored during the testing process. A higher code coverage means a greater probability of triggering vulnerabilities.

The ZFuzz tool performed particularly well in tests conducted on PureFTPD and Tinydtls; in the case of Tinydtls, ZFuzz improved performance by 30.97% compared to AFLNet and 10.93% compared to NSFuzz. For PureFTPD testing, ZFuzz improved by 12.20% and 6.57% respectively. Since the comparison tool NSFuzz also uses a state variable representation of protocol states, it can be concluded that utilizing this state representation scheme effectively enhances code coverage capabilities. However, the experimental results for Live555 indicate that this approach still requires improvement for real-time streaming media transmission protocols.

B. Crash triggering situation

In addition to the commonly used code coverage as an indicator, the number of crash triggers can also directly reflect the tool's effectiveness in vulnerability detection. The table below displays the crash situations of the target protocol during the fuzz testing process, including the number of crashes triggered and the time of the first trigger. When handling crash situations, clustering is performed based on the last instruction address before the crash, followed by manual analysis to identify program vulnerabilities. In practice, the same vulnerability may cause crashes at different instruction addresses, resulting in a higher number of crashes than the number of vulnerabilities.

TABLE III. CRASH TRIGGERING SITUATION

Target Service	AFLNET	NSFUZZ	ZFuzz
Live555	4	4	6
Tinydtls	2	4	7

From table III, it can be seen that compared to the control tools, ZFuzz can trigger more crashes and can cover crashes triggered by other tools. Specifically, the experimental results for Live555 show that AFLNet and NSFuzz each discovered three identical crashes and one unique crash, while ZFuzz completely covered the crashes already discovered by the two tools and also identified one crash that the comparison tools did not find.

In the tests for Tinydtls, the set of crashes discovered by ZFuzz is a subset of the crashes found by NSFuzz, and the crashes found by NSFuzz are a subset of those found by AFLNet. This indicates that the approach proposed by the tool in this article not only covers the crashes discovered by existing tools but also possesses the capability to independently discover new crashes.

TABLE IV. FIRST CRASH TRIGGER TIME

Target Service	AFLNET	NSFUZZ	ZFuzz
Live555	756s	96s	20s
Tinydtls	26s	5s	<1s

In terms of the time taken to trigger crashes for the first time, the tool presented in this article can trigger crashes more quickly. In the tests for Live555, ZFuzz improved performance by 95.50% compared to AFLNet and 64.58% compared to NSFuzz. In the tests for Tinydtls, ZFuzz improved by 92.31% and 60%, respectively, as shown in Table IV.

Through the comparison with NSFuzz and AFLNet, it can be concluded that the state variable representation of protocol states enhances the ability to explore crashes. Additionally, compared to the other two tools, ZFuzz demonstrates that the design of a state machine for protocol states, along with a focus on key states, effectively reduces the time required to discover crashes, highlighting the effectiveness of this approach.

The above experiments indicate that ZFuzz can discover more crashes and trigger them more quickly during fuzz testing, thereby exhibiting better vulnerability excavation capabilities and overall effectiveness.

V. CONCLUSION

This article addresses the issue of inaccurate representation of protocol states by designing a state variable-based expression scheme, which processes the target protocol program through static analysis. It also supports the use of APIs for manual annotation, resulting in a more comprehensive protocol representation scheme and allowing for customizable granularity in state representation.

ZFuzz connects protocol states with the code space, making the information during the testing process more coordinated and effectively utilizing available information, thereby laying the foundational data for state weight calculation and message pool construction. New mutation strategies are designed based on the characteristics of protocol messages, further targeting specific states for testing.

Experiments have demonstrated the feasibility and effectiveness of this approach, achieving greater code coverage and improved crash triggering situations. However, there are still issues related to the coarse granularity during message processing, and further optimization and refinement will continue in the future.

REFERENCES

[1] BEYONDT. Microsoft Vulnerabilities Report 2023[EB/OL]. https://assets.beyondtrust.com/assets/documents/2023-Microsoft-Vulnerability-Report_BeyondTrust.pdf.

[2] The Heartbleed Bug. Retrieved April 10, 2024 from https://heartbleed.com/.

[3] Server Message Block (SMB) Protocol Versions 2 and 3. Retrieved April 10, 2024 from https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-smb/..

[4] WannaCry Ransomware Attack. Retrieved April 10, 2021 from https://en.wikipedia.org/wiki/WannaCry_ransomware_attack.

[5] 360-CERT. NXNS Attack: notification of DNS protocol security vulnerabilities [EB/OL]. https://mp.weixin. qq. com/s/jYxHvIgPyeQknX0dNTZ0Zg. (in Chinese)

[6] MICHAEL E. Peach Fuzzing Platform [EB/OL]. 2021. https://gitlab.com/peachtech/peach-fuzzer-community jtpereyda/boofuzz [EB/OL]. https://github.com/jtpereyda/boofuzz, February 1, 2021.

[7] Joshua Pereyda. 2021. boofuzz: Network Protocol Fuzzing for Humans. Retrieved April 10, 2024 from https://github.com/jtpereyda/boofuzz.

[8] MICHAL Z. AFL (American Fuzzy Lop) [EB/OL]. 2021. https://github.com/google/AFL.

[9] Pham V T, Bohme M, Roychoudhury A. AFLNet: a greybox fuzzer for network protocols[C]//Proc of the 13th International Conference on Software Testing, Validation and Verification. Piscataway, NJ:IEEE Press,2020:460-465.

[10] Roberto Natella. 2021. StateAFL: Greybox fuzzing for stateful network servers. arXiv preprint arXiv:2110.06253 (2021).

[11] Qin S, Hu F, Zhao B, et al. NSFuzz: Towards efficient and state-aware network service fuzzing. ACM Transactions on Software Engineering and Methodology, 2023. [doi:10.1145/3580598]

[12] Greg Banks, Marco Cova, Viktoria Felmetsger, Kevin Almeroth, Richard Kemmerer, and Giovanni Vigna. 2006.SNOOZE: Toward a Stateful Network protocol fuzzer. In International Conference on Information Security. Springer, 343–358.

[13] Humberto J Abdelnur, Radu State, and Olivier Festor. 2007. KiF: A stateful SIP fuzzer. In Proceedings of the 1st International Conference on Principles, Systems and Applications of IP Telecommunications. 47–56.

[14] Serge Gorbunov and Arnold Rosenbloom. 2010. Autofuzz: Automated network protocol fuzzing framework. IJCSNS 10, 8 (2010), 239

[15] Takahisa Kitagawa, Miyuki Hanaoka, and Kenji Kono. 2010. AspFuzz: A state-aware protocol fuzzer based on application-layer protocols. In IEEE Symposium on Computers and Communications. IEEE, 202–208.

[16] Hugo Gascon, Christian Wressnegger, Fabian Yamaguchi, Daniel Arp, and Konrad Rieck. 2015. Pulsar: Stateful blackbox fuzzing of proprietary network protocols. In International Conference on Security and Privacy in Communication Systems. Springer, 330–347

[17] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. 2022. Stateful greybox fuzzing. (2022). arXiv:2204.02545 http://arxiv.org/abs/2204.02545.

[18] LI J Q, LI S Y, SUN G, et al. SNPSFuzzer: a fast greybox fuzzer for stateful network protocols using snapshots [EB/OL]. https://arxiv. org/abs/2202. 03643. pdf.